אלגוריתמים – מבחן מסכם.

פרופ' תמי תמיר

21.2.2022 – מועד ב'

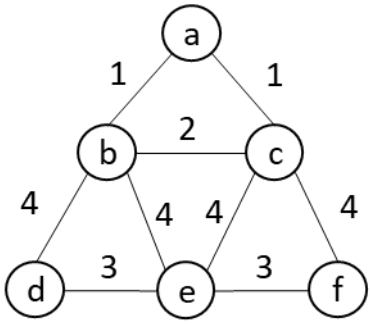
ת.ז:_____

- משך המבחן: 3 שעות.
- מותר דף עזר אחד.
- מותר להשתמש בדפי טיוטה אך יש לענות רק על דפי הבחינה, במקום המוקצב לכך בכל שאלה. שימו לב שרק הצד הקידמי בכל דף נסרק ומועבר לבדיקה.
- אין לכתוב בצד השני של הדף, אם יש צורך, ניתן להשתמש בדף הנוסף שבסוף השאלון.
- מותר לצטט ולהסתמך (מבלי להוכיח) על משפטים וטענות שהוכחו בכתה או בתרגילי הבית.
- בכל שאלה או סעיף שמופיע לידם סימן ♣, ניתן לכתוב 'לא יודע' ולקבל רבע מהניקוד (ערך תחתון).

ב ה צ ל ח ה !

<u>שאלה 1 (20 נק')</u>

♣ **1.1** (10 נק') השאלה מתייחסת לגרף שבאיור.
אין צורך לנמק את תשובתכם לסעיפים הבאים.



1. מהו מחיר כל **עץ פורש מינימום** בגרף? 12
2. כמה עפ"מים שונים יש לגרף? 4
3. מריצים את אלגוריתם **קרוסקל** על הגרף. מה יהיו מחלקות ה-union-find אחרי שהאלג' מתייחס לכל הקשתות שמחירן נמוך מ-4.

{a,b,c}, {d,e,f}

4. ציינו קשת אחת שיכולה להיות הקשת **השלישית** שתצטרף לעפ"מ בריצת אלג' **פרים** המתחילה בצומת c.

bd

Any edge of cost 4

ab

5. ציינו קשת אחת שיכולה להיות הקשת **הרביעית** שתצטרף לעפ"מ בריצת אלג' **פרים** המתחילה בצומת e.

Any edge of cost 1

♣ **1.2** (10 נק') נתון גרף קשיר פשוט ולא מכוון $G = (V, E)$, עם מחירים שלמים חיוביים על הקשתות, ומספר שלם $c \in \mathbb{N}$. נסמן ב-A את קבוצת כל קשתות הגרף שמחירן c. הציעו אלגוריתם **יעיל** עבור הבעיה הבאה: בכמה ירד מחיר עפ"מ בגרף אם המחיר של כל אחת מקשתות A יוזל ב-1 (כלומר מחיר כל אחת מהן ירד להיות $c - 1$)?
תארו את האלגוריתם, נמקו בקצרה את הנכונות (אין צורך להוכיח פורמלית, רק לציין את הרעיונות המרכזיים בהוכחה), ונתחו את סיבוכיות הזמן.

1. Remove from G all edges having cost at least c
2. Let x be the number of components in the resulting graph.
3. Add back the edges of A
4. Let y be the number of components in the resulting graph.
5. Return x-y

Correctness: Every sort of the edges according to the original costs, is a valid sort of the edges after the reduction, therefore, a run of Kruskal on both lists will produce the same MST. Therefore, the question is actually how many edges from A participate in every MST of G. Assume that Kruskal's algorithm is performed on the graph. x is the number of U/F sets just before the edges of A are considered, y is the number of components after the edges of A are considered. x-y is exactly the number of edges having cost c in every MST of the graph.

Complexity:
Step 1,3: O(|V|+|E|)
Steps 2,4: O(|V|+|E|) – using DFS
Step 5: O(1)
Total: O(|V|+|E|)=O(|E|)

שאלה 2 (18 נק')

תהי Names קבוצה של מחרוזות המהוות שמות של סטודנטים. תהי $A=a_1,a_2,\ldots,a_n$ סדרה של n תוים (שימו לב שהאינדקס הראשון הוא 1).
עליכם להכריע האם ניתן לחלק את הסדרה A לתתי סדרות רציפות, שכל אחת מהן מהווה מחרוזת השייכת ל-Names. למשל עבור Names ={din, dan, dana, dean, adnan, adina, diana, dina, andy}, n=12, A=danadnandean , התשובה היא True, מכיוון שניתן לחלק את A ל- dan, adnan, dean, שכולן מחרוזות השייכות ל-Names.

לכל $1\le k\le j\le n$ נניח שניתן לקרוא לפונק IsStudent(k,j) ולדעת בזמן O(1) האם תת הסידרה של A המורכבת מרצף התווים $a_k,..,a_j$ מהווה מחרוזת ששייכת ל-Names.
למשל בדוגמה, IsStudent(1,4)=True מכיוון שהמחרוזת dana שייכת ל-Names.

פיתרו את הבעיה בעזרת **תכנות דינאמי**. לכל $1\le i\le n$, יהי SplitPrefix[i] משתנה בינארי המציין האם ניתן לחלק את תת-הסדרה $a_1,\ldots a_i$ לתתי סדרות המהוות מחרוזות השייכות ל-Names.

2.1 (2 נק') מהו הביטוי המתאר את ערך הפתרון (אין צורך להסביר)?

SplitPrefix[n]

♣ **2.2** (12 נק') כתבו נוסחת תכנות דינאמי לחישוב SplitPrefix[i]. הפרידו בין הבסיס למקרה הכללי. הסבירו את הנוסחה במילים.

Base case: SplitPrefix[0] = True

For $1\le i\le n$, SplitPrefix[i] = OR $_{1\le k\le i}$ (IsStudent(k,i) and SplitPrefix[k−1])

The sub-sequence $a_1,\ldots a_i$ can be divided into sub-sequences which are strings in Names, if it has some suffix $a_k,\ldots a_i$ which is a name, and $a_1,\ldots a_{k-1}$ can be divided into sub-sequences which are strings in Names. We check all possible choices of k.

For the general case, we can also use:

For $1\le i\le n$, SplitPrefix[i] = OR $_{name\in Names}$ (IsStudent(i−|name|+1,i) and SplitPrefix[i−|name|])

In case |Names|<n, this version improves the time complexity from $O(n^2)$ to $O(n\cdot|Names|)$ (see next item).

Note that both versions can be expressed with a ∃ quantifier instead of the OR operator,

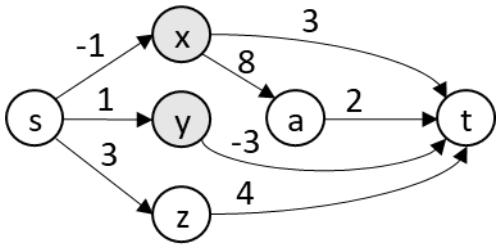
♣ **2.3** (4 נק') תארו את טבלת התכנות הדינאמי, מה גודלה, ומה סדר מילוי הערכים בטבלה. נתחו את סיבוכיות הזמן של האלגוריתם.

The table has one dimension and size n+1 (if the base case is part of the table).
To fill entry i, we need to consider all the entries 1..i. For every candidate partition, the calculation takes O(1), and there are i possible partitions. Therefore, calculating SplitPrefix[i] takes O(i) and the whole table can be filled from entry 0 to entry n in time $O(n^2)$.

♣ שאלה 3 (14 נק')

נתון גרף $G = (V, E)$, מכוון וחסר מעגלים מכוונים (DAG), עם מחירים כלשהם על הקשתות. בנוסף נתונים שלושה צמתים s, a, t וקבוצת צמתים $W \subseteq V$. ידוע ש- $\{s, a, t\} \cap W = \emptyset$. הציעו אלגוריתם יעיל המוצא את מחיר המסלול הזול ביותר מ-s ל-t תחת המגבלה הבאה: אם המסלול עובר בצומת מ-W, אז הוא חייב לעבור בהמשך, לפני ההגעה ל-t, בצומת a.
הבהרה: אם המסלול עובר במספר צמתים מ-W , עליו לעבור בצומת a אחרי המעבר האחרון בצומת מ-W .

תארו את האלגוריתם במילים ובאופן פורמלי, **אין צורך להוכיח**, ציינו ונמקו את **סיבוכיות הזמן**.



דוגמה: בגרף שבאיור, עבור W={x,y}, המסלול הזול ביותר מ-s ל-t הוא המסלול s-z-t שמחירו 7. אין מסלול חוקי שעובר בצומת y, והמסלול החוקי הזול ביותר שעובר בצומת x הוא s-x-a-t שמחירו 9. המסלול s-x-t שמחירו 2 אינו חוקי.

Algorithm:
1. Construct a DAG G'=(V', E').
V'= V1 ∪V2, where each of V1, V2 is a copy of V.
E' = {(v1,u1) | (v,u)∈E} ∪ {(v2, u2)| (v,u)∈E, v∉W} ∪ {(s1,s2), (a1,a2)}.
The weight of an edge e in each copy (of type (v1,u1) or (v2,u2)) is w(e). The weight of the two crossing edges is 0.
In words: The graph G' consists of two copies of G. In the second copy there are no edges going out from the vertices of W. The two edges between the copies enable a free pass from s1 to s2 and from a1 to a2.
2. Run the algorithm for shortest path in a DAG on G' with vertex s1 as a source.
3. Return d(s1,t2)

Note that G' is a DAG because each copy is a DAG and the added edges are from the first copy to the second, therefore, they agree with any topological sort in which all the vertices of V1 are before all the vertices of V2.

Correctness idea (not required): There are two ways to reach t2, Either by a path that begins in (s1,s2) followed by a path that does not pass in W, or by a path that begins in V1, and moves to V2 along the edge (a1,a2). In this case, the path can visit vertices from W before this edge.

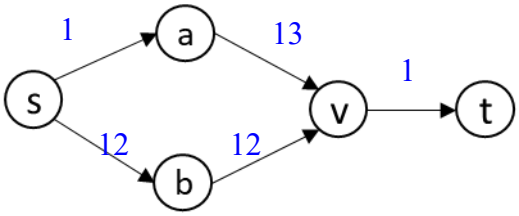
Time Complexity:
1. Building G': O(2|E|+2|V|) = O(|E|+|V|)
2. Running DAG alg on G': O(2|E|+2|V|) = O(|E|+|V|)
3. Output: O(1)
All together: O(|E|+|V|)

שאלה 4 (20 נקודות)

נתונה רשת זרימה. באפשרותכם לבחור שתי קשתות $e_1, e_2 \in E$, **ולהחליף** בין הקיבולים שלהן.

♣ **4.1** (5 נק') באיור נתונה **רשת זרימה**. קבעו קיבולים לקשתות כך שיתקיים התיאור הבא: ערך זרימת המקסימום ברשת הוא f* וקיים זוג קשתות שאם נחליף בין הקיבולים שלהן נקבל רשת זרימה שערך זרימת המקסימום בה הוא 13f* (כלומר, החלפה בין הקיבולים של זוג קשתות מגדילה את ערך זרימת המקסימום פי 13).

ציינו את הקיבולים באיור, והשלימו:



ערך זרימת המקסימום ברשת המקורית:

1

זוג הקשתות שהחלפת הקיבולים שלהן תביא להגדלת ערך הזרימה פי 13:

(a,v), (v,t)

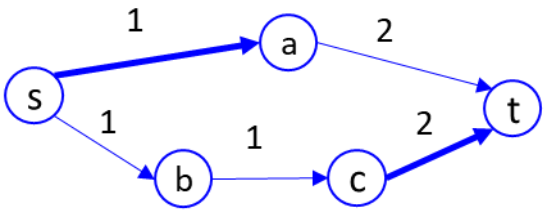
♣ **4.2** (5*3 = 15 נק') תהי G רשת זרימה. **הוכיחו או הפריכו** כל אחת מהטענות הבאות (בנפרד):

א. ♣ אם לכל הקשתות ב-G קיבולים שונים, כלומר, לכל $e_1, e_2 \in E$ מתקיים ש-$c(e_1) \neq c(e_2)$, אז בהכרח קיים זוג קשתות שהחלפה בין הקיבולים שלהן משפרת את ערך זרימת המקסימום.

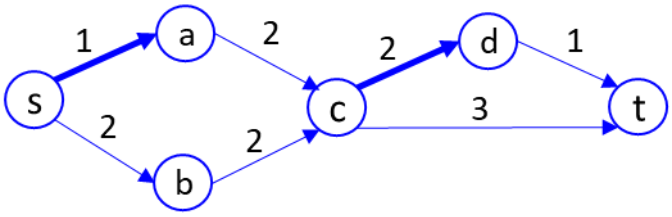
False: consider s→ a → t with any distinct capacities. The flow is min(c1,c2) before and after the switch.

ב. ♣ אם קיים ב-G זוג קשתות שהחלפה בין הקיבולים שלהן משפרת את ערך זרימת המקסימום, אז יש ב-G (לפני ההחלפה) חתך מינימום יחיד.

False: In this network f*=2, both {s} and {s,b} are min cuts.
Switching the capacities of the bold edges results in a network with f*=3.



Another example (here f* increases from 3 to 4):



ג. ♣ אם החלפה בין הקיבולים של e_1, e_2 מביאה להגדלה של ערך זרימת המקסימום ב-G, אז לכל חתך מינימום $(S, V \setminus S)$ ב- G (לפני ההחלפה), קבוצת הקשתות $\mathrm{OUT}(S)$ כוללת בדיוק קשת אחת מתוך e_1, e_2.

Proof: Let $(S, V \setminus S)$ be a min cut in the original network, if it does not include any of e_1 and e_2 or if it includes both, then the total capacity of edges in OUT(S) does not change by the switch. Therefore, the cut-capacity remains the same and the value of the max-flow cannot increase.

שאלה 5 (16 נק')

♣ **5.1** (6 נק') יהי $G = (V, E)$ גרף פשוט, קשיר, ולא מכוון. **הוכיחו או הפריכו:**
קיים צומת $v \in V$ כך שהגרף המתקבל מהסרת v והקשתות הסמוכות אליו הוא קשיר.

The case |V|≤1 is trivial. Otherwise, let T be a spanning tree of G. Let v be a leaf in T (there are at least two leaves). When we remove v and the (only) edge adjacent to it in T, we remain with a tree. Hence $G' = (V - \{v\}, E - E_v)$, where E_v is the set of edges adjacent to v, has a spanning tree, and in particular it is connected.

Common mistake: many students argued that if $\deg(u) \geq 2$ for all $u \in V$, then there exists a cycle, in which case some vertex v in the cycle (usually arbitrary or minimal-degree) satisfies the requirement. This argument is incorrect. Counter example: Take two disjoint simple cycles of length $n \geq 6$. Choose two vertices v_1 and v_2, one from each cycle, and add the edge (v_1, v_2). For each cycle, turn the remaining $n - 1$ vertices into a clique by adding the necessary edges. Note that v_1, for example, is a minimal-degree vertex in some cycle, yet removing it and its adjacent edges yields a disconnected graph.

♣ **5.2** (10 נק') יהי $G = (V, E)$ גרף פשוט, קשיר, ולא מכוון. נתון ש- $|V| = 100, |E| = 100$ וש-G **אינו** C_{100} (מעגל בן 100 צמתים). השתמשו בעקרון שובך היונים כדי **להוכיח** שקיימים שני צמתים $a, b \in V$ כך שהעצים הפורשים הלא מכוונים המתקבלים מ-$DFS(a)$ ו-$DFS(b)$ הם זהים.

עקרון שובך היונים: לכל $1 \leq y < x$, אם נשים x יונים ב-y שובכים, אז לפחות שובך אחד יכיל לפחות שתי יונים.

The graph G is a tree with one additional edge. Therefore, it includes exactly one cycle C. Since the graph is not C_{100}, the cycle consists of at most 99 edges. Every spanning tree of G includes all but a single edge from C, therefore, G has |C| ≤ 99 different spanning trees.
Every run of DFS returns one of these spanning trees. Since there are 100 vertices, and at most 99 possible outputs of DFS, for at least two vertices, DFS returns the same ST.



♣ **<u>שאלה 6 (12 נק')</u>**

נתונים n תפוחים ו-m בננות. לתפוח ה-i משקל a_i. לבננה ה-i משקל b_i. בנוסף נתון מספר H.
ערן צריך להכין מארזים שכוללים תפוח אחד ובננה אחת. מארז הוא *תקין* אם המשקל הכולל של הפירות שבו הוא לפחות H. כלומר, יש בו תפוח אחד $1 \le i \le n$ ובננה אחת $1 \le j \le m$, כך ש-$a_i + b_j \ge H$.
הציעו אלגוריתם חמדן יעיל הקובע כיצד לארוז את הפירות כך שמספר המארזים התקינים יהיה גבוה ככל שניתן.

♣ **6.1** (6 נק') תארו את האלגוריתם (אין צורך לנתח סיבוכיות).

Algorithm:

1. Sort the apples a1 ≥ a2 ≥ …≥ an
2. Sort the bananas b1 ≥ b2 ≥ …≥ bm
3. Repeat as long as n≥1 and m≥1
    a. If a1+bm ≥ W then pair (A1, Bm). Remove the largest apple and the smallest banana (reduce n and m by 1 and re-index the remaining apples accordingly)
    b. Else, remove the smallest banana. Reduce m by 1.

♣**6.2** (6 נק') כתבו את טענת ה-**greedy choice property** המתאימה (אין צורך להוכיח).

Lemma 1: if a1+bm < W then there is no solution in which the m-th banana is paired.
Lemma 2 : If a1+bm ≥ W then exists an optimal solution with the pair (A1, Bm).

Proof (not needed)
1: For every apple, i, ai+bm ≤ a1+bm <W, therefore bm is not part of a valid pair.
2: Consider a solution S. If both A1 and Bm are not paired, then S can be improved.
If only one of them is paired, its matching fruit can be replaced, to get an alternative valid pair (A1,Bm).
Assume both are paired {(A1,Bj), (Ai,Bm)}⊆S.
Both pairs are valid so a1+bj ≥ W and ai+bm ≥ W.
Exchange the the pairs to get (A1,Bm), (Ai,Bj). The first pair is valid (this is the condition in Lemma 2). The second pair is valid since ai+bj ≥ ai+bm ≥ W.